

Loja de Eletrônicos

Relatório técnico da implementação em Java

Aluno	Carlos Eduardo Michelle Marques
Código de matrícula	842844
Repositório	https://github.com/yKdu7z/loja-eletronica
Entrega	Aplicação Java + relatório técnico em PDF

Este relatório descreve uma aplicação de loja de eletrônicos baseada em orientação a objetos. O sistema organiza produtos por especialização, permite que o cliente adicione itens ao carrinho em quantidades variadas e calcula o total da compra.

1. Verificação dos requisitos

O código foi revisado em relação ao enunciado e atende aos pontos solicitados para a atividade.

Requisito	Atendimento	Evidência no código
Classe base Produto	Atendido	Produto é abstrata e concentra marca, modelo e preço.
Smartphone, Laptop e TV	Atendido	As três classes herdam de Produto e possuem atributos próprios.
Cliente	Atendido	Cliente mantém dados básicos e seu carrinho de compras.
Carrinho de compras	Atendido	CarrinhoDeCompras armazena itens, quantidades e calcula o total.
Quantidades variadas	Atendido	ItemCarrinho controla quantidade e subtotal por produto.
Total a pagar	Atendido	visualizarTotalCarrinho() retorna o total formatado.

2. Execução da aplicação

A execução demonstra o cadastro de três produtos, a inclusão no carrinho e o cálculo do total a pagar pelo cliente Carlos Eduardo Michelle Marques.

```
PS D:\wamp64\www\TRABALHO POO> java -cp out br.com.lojaeletronica.Main
```

```
=== Catálogo da Loja de Eletrônicos ===
```

```
Smartphone | Samsung Galaxy S24 | R$ 4.299,90 | 256GB armazenamento, 8GB RAM, tela 6.2", bateria 4000mAh
```

```
Laptop | Dell Inspiron 15 | R$ 3.899,00 | Intel Core i7, 16GB RAM, SSD 512GB, video Intel Iris Xe
```

```
TV | LG OLED C3 | R$ 5.499,99 | 55", 4K UHD, OLED, Smart TV
```

```
=== Carrinho de Compras ===
```

```
Cliente: Carlos Eduardo Michelle Marques
```

```
2x Smartphone Samsung Galaxy S24 - subtotal: R$ 8.599,80
```

```
1x Laptop Dell Inspiron 15 - subtotal: R$ 3.899,00
```

```
1x TV LG OLED C3 - subtotal: R$ 5.499,99
```

```
Total a pagar: R$ 17.998,79
```

3. Código-fonte

A seguir está o código-fonte completo da aplicação.

src/br/com/lojaeletronica/Produto.java

```
package br.com.lojaeletronica;

import java.math.BigDecimal;
import java.text.NumberFormat;
import java.util.Locale;
import java.util.Objects;

public abstract class Produto {
    private final String marca;
    private final String modelo;
    private final BigDecimal preco;

    protected Produto(String marca, String modelo, BigDecimal preco) {
        if (preco == null || preco.compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("O preço do produto não pode ser negativo.");
        }

        this.marca = validarTexto(marca, "marca");
        this.modelo = validarTexto(modelo, "modelo");
        this.preco = preco;
    }

    public String getMarca() {
        return marca;
    }

    public String getModelo() {
        return modelo;
    }

    public BigDecimal getPreco() {
        return preco;
    }

    public String getPrecoFormatado() {
        NumberFormat formatador = NumberFormat.getCurrencyInstance(Locale.forLanguageTag("pt-BR"));
        return formatador.format(preco);
    }

    public abstract String getCategoria();

    public abstract String getEspecificacoesTecnicas();

    public String exibirDetalhes() {
        return getCategoria() + " | " + marca + " " + modelo + " | " + getPrecoFormatado()
            + " | " + getEspecificacoesTecnicas();
    }

    private static String validarTexto(String valor, String campo) {
        if (valor == null || valor.isBlank()) {
            throw new IllegalArgumentException("O campo " + campo + " é obrigatório.");
        }

        return valor.trim();
    }

    @Override
    public String toString() {
        return getCategoria() + " " + marca + " " + modelo;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }

        if (!(obj instanceof Produto outroProduto)) {
            return false;
        }

        return marca.equalsIgnoreCase(outroProduto.marca)
            && modelo.equalsIgnoreCase(outroProduto.modelo)
            && getCategoria().equalsIgnoreCase(outroProduto.getCategoria());
    }

    @Override
    public int hashCode() {
        return Objects.hash(marca.toLowerCase(Locale.ROOT), modelo.toLowerCase(Locale.ROOT), getCategoria());
    }
}
```

src/br/com/lojaeletronica/Smartphone.java

```
package br.com.lojaeletronica;

import java.math.BigDecimal;

public class Smartphone extends Produto {
    private final int armazenamentoGb;
    private final int memoriaRamGb;
    private final double tamanhoTelaPolegadas;
    private final int capacidadeBateriaMah;

    public Smartphone(
        String marca,
        String modelo,
        BigDecimal preco,
        int armazenamentoGb,
        int memoriaRamGb,
        double tamanhoTelaPolegadas,
        int capacidadeBateriaMah) {
        super(marca, modelo, preco);
        this.armazenamentoGb = validarPositivo(armazenamentoGb, "armazenamento");
        this.memoriaRamGb = validarPositivo(memoriaRamGb, "memoria RAM");
        this.tamanhoTelaPolegadas = validarPositivo(tamanhoTelaPolegadas, "tamanho da tela");
        this.capacidadeBateriaMah = validarPositivo(capacidadeBateriaMah, "capacidade da bateria");
    }

    @Override
    public String getCategoria() {
        return "Smartphone";
    }

    @Override
    public String getEspecificacoesTecnicas() {
        return armazenamentoGb + "GB armazenamento, " + memoriaRamGb + "GB RAM, tela "
            + tamanhoTelaPolegadas + "\"", bateria " + capacidadeBateriaMah + "mAh";
    }

    private static int validarPositivo(int valor, String campo) {
        if (valor <= 0) {
            throw new IllegalArgumentException("O campo " + campo + " deve ser maior que zero.");
        }

        return valor;
    }

    private static double validarPositivo(double valor, String campo) {
        if (valor <= 0) {
            throw new IllegalArgumentException("O campo " + campo + " deve ser maior que zero.");
        }

        return valor;
    }
}
```

src/br/com/lojaeletronica/Laptop.java

```
package br.com.lojaeletronica;

import java.math.BigDecimal;

public class Laptop extends Produto {
    private final String processador;
    private final int memoriaRamGb;
    private final int armazenamentoSsdGb;
    private final String placaDeVideo;

    public Laptop(
        String marca,
        String modelo,
        BigDecimal preco,
        String processador,
        int memoriaRamGb,
        int armazenamentoSsdGb,
        String placaDeVideo) {
        super(marca, modelo, preco);
        this.processador = validarTexto(processador, "processador");
        this.memoriaRamGb = validarPositivo(memoriaRamGb, "memoria RAM");
        this.armazenamentoSsdGb = validarPositivo(armazenamentoSsdGb, "armazenamento SSD");
        this.placaDeVideo = validarTexto(placaDeVideo, "placa de vídeo");
    }

    @Override
    public String getCategoria() {
        return "Laptop";
    }

    @Override
    public String getEspecificacoesTecnicas() {
        return processador + ", " + memoriaRamGb + " GB RAM, SSD " + armazenamentoSsdGb
            + " GB, vídeo " + placaDeVideo;
    }

    private static String validarTexto(String valor, String campo) {
        if (valor == null || valor.isBlank()) {
            throw new IllegalArgumentException("O campo " + campo + " é obrigatório.");
        }

        return valor.trim();
    }

    private static int validarPositivo(int valor, String campo) {
        if (valor <= 0) {
            throw new IllegalArgumentException("O campo " + campo + " deve ser maior que zero.");
        }

        return valor;
    }
}
```

src/br/com/lojaeletronica/TV.java

```
package br.com.lojaeletronica;

import java.math.BigDecimal;

public class TV extends Produto {
    private final int tamanhoPolegadas;
    private final String resolucao;
    private final boolean smartTv;
    private final String tecnologiaTela;

    public TV(
        String marca,
        String modelo,
        BigDecimal preco,
        int tamanhoPolegadas,
        String resolucao,
        boolean smartTv,
        String tecnologiaTela) {
        super(marca, modelo, preco);
        this.tamanhoPolegadas = validarPositivo(tamanhoPolegadas, "tamanho da TV");
        this.resolucao = validarTexto(resolucao, "resolução");
        this.smartTv = smartTv;
        this.tecnologiaTela = validarTexto(tecnologiaTela, "tecnologia da tela");
    }

    @Override
    public String getCategoria() {
        return "TV";
    }

    @Override
    public String getEspecificacoesTecnicas() {
        String conectividade = smartTv ? "Smart TV" : "TV convencional";
        return tamanhoPolegadas + "\", " + resolucao + "\", " + tecnologiaTela + "\", " + conectividade;
    }

    private static String validarTexto(String valor, String campo) {
        if (valor == null || valor.isBlank()) {
            throw new IllegalArgumentException("O campo " + campo + " é obrigatório.");
        }

        return valor.trim();
    }

    private static int validarPositivo(int valor, String campo) {
        if (valor <= 0) {
            throw new IllegalArgumentException("O campo " + campo + " deve ser maior que zero.");
        }

        return valor;
    }
}
```

src/br/com/lojaeletronica/ItemCarrinho.java

```
package br.com.lojaeletronica;

import java.math.BigDecimal;

public class ItemCarrinho {
    private final Produto produto;
    private int quantidade;

    public ItemCarrinho(Produto produto, int quantidade) {
        if (produto == null) {
            throw new IllegalArgumentException("O produto é obrigatório.");
        }

        this.produto = produto;
        alterarQuantidade(quantidade);
    }

    public Produto getProduto() {
        return produto;
    }

    public int getQuantidade() {
        return quantidade;
    }

    public void adicionarQuantidade(int quantidade) {
        validarQuantidade(quantidade);
        this.quantidade += quantidade;
    }

    public void alterarQuantidade(int quantidade) {
        validarQuantidade(quantidade);
        this.quantidade = quantidade;
    }

    public BigDecimal getSubtotal() {
        return produto.getPreco().multiply(BigDecimal.valueOf(quantidade));
    }

    private static void validarQuantidade(int quantidade) {
        if (quantidade <= 0) {
            throw new IllegalArgumentException("A quantidade deve ser maior que zero.");
        }
    }
}
```

src/br/com/lojaeletronica/CarrinhoDeCompras.java

```
package br.com.lojaeletronica;

import java.math.BigDecimal;
import java.text.NumberFormat;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Optional;

public class CarrinhoDeCompras {
    private final List<ItemCarrinho> itens = new ArrayList<>();

    public void adicionarProduto(Produto produto, int quantidade) {
        Optional<ItemCarrinho> itemExistente = buscarItem(produto);

        if (itemExistente.isPresent()) {
            itemExistente.get().adicionarQuantidade(quantidade);
            return;
        }

        itens.add(new ItemCarrinho(produto, quantidade));
    }

    public void removerProduto(Produto produto) {
        itens.removeIf(item -> item.getProduto().equals(produto));
    }

    public List<ItemCarrinho> getItens() {
        return Collections.unmodifiableList(itens);
    }

    public BigDecimal calcularTotal() {
        return itens.stream()
            .map(ItemCarrinho::getSubtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }

    public String getTotalFormatado() {
        NumberFormat formatador = NumberFormat.getCurrencyInstance(Locale.forLanguageTag("pt-BR"));
        return formatador.format(calcularTotal());
    }

    public boolean estaVazio() {
        return itens.isEmpty();
    }

    private Optional<ItemCarrinho> buscarItem(Produto produto) {
        return itens.stream()
            .filter(item -> item.getProduto().equals(produto))
            .findFirst();
    }
}
```


src/br/com/lojaeletronica/Cliente.java

```
package br.com.lojaeletronica;

public class Cliente {
    private final String nome;
    private final String cpf;
    private final CarrinhoDeCompras carrinhoDeCompras;

    public Cliente(String nome, String cpf) {
        this.nome = validarTexto(nome, "nome");
        this.cpf = validarTexto(cpf, "CPF");
        this.carrinhoDeCompras = new CarrinhoDeCompras();
    }

    public String getNome() {
        return nome;
    }

    public String getCpf() {
        return cpf;
    }

    public CarrinhoDeCompras getCarrinhoDeCompras() {
        return carrinhoDeCompras;
    }

    public void adicionarProdutoAoCarrinho(Produto produto, int quantidade) {
        carrinhoDeCompras.adicionarProduto(produto, quantidade);
    }

    public String visualizarTotalCarrinho() {
        return carrinhoDeCompras.getTotalFormatado();
    }

    private static String validarTexto(String valor, String campo) {
        if (valor == null || valor.isBlank()) {
            throw new IllegalArgumentException("O campo " + campo + " é obrigatório.");
        }

        return valor.trim();
    }
}
```

src/br/com/lojaeletronica/Main.java

```
package br.com.lojaeletronica;

import java.math.BigDecimal;
import java.text.NumberFormat;
import java.util.Locale;

public class Main {
    public static void main(String[] args) {
        Produto smartphone = new Smartphone(
            "Samsung",
            "Galaxy S24",
            new BigDecimal("4299.90"),
            256,
            8,
            6.2,
            4000);

        Produto laptop = new Laptop(
            "Dell",
            "Inspiron 15",
            new BigDecimal("3899.00"),
            "Intel Core i7",
            16,
            512,
            "Intel Iris Xe");

        Produto tv = new TV(
            "LG",
            "OLED C3",
            new BigDecimal("5499.99"),
            55,
            "4K UHD",
            true,
            "OLED");

        Cliente cliente = new Cliente("Carlos Eduardo Michelle Marques", "123.456.789-00");

        cliente.adicionarProdutoAoCarrinho(smartphone, 2);
        cliente.adicionarProdutoAoCarrinho(laptop, 1);
        cliente.adicionarProdutoAoCarrinho(tv, 1);

        exibirCatalogo(smartphone, laptop, tv);
        exibirCarrinho(cliente);
    }

    private static void exibirCatalogo(Produto... produtos) {
        System.out.println("=== Catálogo da Loja de Eletrônicos ===");
        for (Produto produto : produtos) {
            System.out.println(produto.exibirDetalhes());
        }
        System.out.println();
    }

    private static void exibirCarrinho(Cliente cliente) {
        NumberFormat formatador = NumberFormat.getCurrencyInstance(Locale.forLanguageTag("pt-BR"));

        System.out.println("=== Carrinho de Compras ===");
        System.out.println("Cliente: " + cliente.getNome());

        for (ItemCarrinho item : cliente.getCarrinhoDeCompras().getItens()) {
            System.out.println(item.getQuantidade() + "x " + item.getProduto()
                + " - subtotal: " + formatador.format(item.getSubtotal()));
        }

        System.out.println("Total a pagar: " + cliente.visualizarTotalCarrinho());
    }
}
```